Factory Method

ry Method · Create Email/SMS/Push notifiers without client coupling · Build with W

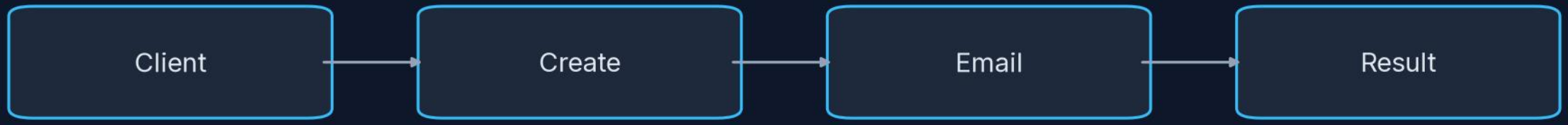
Today's Hook

- Problem: Create Email/SMS/Push notifiers without client coupling
- Pattern: Factory Method — earned from the pain, not trivia.
- Runnable main on JDK 25 with captured stdout.
- Naive → solution → failure paths → trade-offs.

The Problem

- Create Email/SMS/Push notifiers without client coupling
- Naive design shows the cost clearly.
- Pattern isolates the force behind a clean boundary.
- Tests cover happy path and injected failures.

Structure



Factory Method: Create Email/SMS/Push notifiers without client cou

Real-World Scenario

- Running example: Create Email/SMS/Push notifiers without client coupling
- Same names in naive code, refactor, and Demo output.
- No shared StreamForge framing unless it fits.
- Product language first; pattern vocabulary second.

Naive Code

```java
package com.buildwithwaqar.patterns.gof.creational.factorymethod.problem;

/** Naive: client switches on channel strings and news concrete types. */
public final class NaiveNotify {
  public static String send(String channel, String to, String body) {
    if ("email".equals(channel)) return "email:" + to + ":" + body;
    if ("sms".equals(channel)) return "sms:" + to + ":" + body;
    if ("push".equals(channel)) return "push:" + to + ":" + body;
    throw new IllegalArgumentException("unknown channel " + channel);
  }
}
```

Why Naive Hurts

- Call sites couple to the wrong shape or leak on partial failure.
- Parameter drilling / duplicated envelopes / missing rollback.
- Hard to unit-test without the real dependency.
- New variants require shotgun edits.

Solution

```
package com.buildwithwaqar.patterns.gof.creational.factorymethod.problem;
public record EmailNotifier() implements Notifier {
  public String send(String to, String body) { return "email:" + to + ":" + body; }
}


package com.buildwithwaqar.patterns.gof.creational.factorymethod.problem;
public record PushNotifier() implements Notifier {
  public String send(String to, String body) { return "push:" + to + ":" + body; }
}
```

Runnable main

```java
package com.buildwithwaqar.patterns.gof.creational.factorymethod.problem;

public final class ProblemDemo {
  public static void main(String[] args) {
    System.out.println("=== Factory Method: Email/SMS/Push notifiers ===");
    System.out.println("--- naive ---");
    System.out.println(NaiveNotify.send("email", "a@b.c", "hi"));
    try { NaiveNotify.send("fax", "a", "x"); } catch (IllegalArgumentException e) { System.out.println("caught=" + e.getMessage()); }
    System.out.println("--- factory method ---");
    NotifierFactory email = new EmailFactory();
    NotifierFactory sms = new SmsFactory();
    System.out.println(email.notifyUser("a@b.c", "hi"));
    System.out.println(sms.notifyUser("+1", "hi"));
    System.out.println(new PushFactory().notifyUser("device", "hi"));
  }
}
```

Walkthrough Output

```
=== Factory Method: Email/SMS/Push notifiers ===
--- naive ---
email:a@b.c:hi
caught=unknown channel fax
--- factory method ---
email:a@b.c:hi
sms:+1:hi
push:device:hi
```

Step-by-Step Trace

- Each stdout line maps to a stage above.
- Re-run the main class locally on JDK 25.
- Failure-path lines prove compensations / rejects / missing files.
- Pause and predict the next line before revealing.

Modern Java 25 / Zazr

```
// Java 25 / Zazr direction
// records + sealed + switch
// inject Env / collaborators at the edge
// virtual threads for blocking fan-out
```

Trade-offs

- Clarity vs ceremony — refuse five types for two lines.
- Teaching demos omit production metrics/idempotency keys.
- Prefer simpler language features when they erase the force.
- Know the twin pattern often confused with Factory Method.

Related Patterns

- Contrast Factory Method with its neighbor pattern.
- Name the roles from memory in sixty seconds.
- Point to the season map for what comes next.
- Link the GitHub episode folder in the description.

Recap

- Problem → naive → Factory Method → Demo stdout.
- Failure paths are part of the lesson.
- Teach when NOT to use it.
- Practice: break a test, fix it, re-run the main.